API Specification: Authentication & Security Protocols

1. Authentication Overview

The system uses token-based authentication via JSON Web Tokens (JWT) for secure and stateless communication. Users receive access and refresh tokens upon successful login, ensuring authenticated access to protected endpoints.

Example token header:

- Authorization: Bearer <access_token>

Token Expiration:

• Access Token: 1 hour
• Refresh Token: 7 days

2. User Roles and Access Levels

Role	Permissions
Student/Staff	View available equipment and facilities, submit requests, check status
Administrator	Approve/decline requests, manage availability, view reports
Super Admin	Manage users, configure policies, handle backups

3. Security Protocols

- Passwords hashed using bcrypt before storage.
- HTTPS enforced for all communications.
- Input validation and sanitization to prevent SQL injection, XSS, and CSRF.
- Role-Based Access Control (RBAC) for different access levels.
- Tokens stored securely and invalidated upon logout.
- Audit logging for all login and administrative actions.

4. Example Endpoints

Method	Endpoint	Description	Auth Required
POST	/api/auth/register	Create new user account	No
POST	/api/auth/login	Authenticate user	No

		and return JWT	
POST	/api/auth/refresh	Get new access token using refresh token	Yes
GET	/api/auth/logout	Invalidate tokens and end session	Yes
GET	/api/user/profile	Retrieve authenticated user details	Yes
PATCH	/api/user/update	Update user information	Yes

Explanation of Authentication Requirements

The register and login endpoints do not require authentication because they serve as entry points for users who have not yet logged in or created an account. These endpoints allow users to register and authenticate, after which they receive JWT tokens. Subsequent requests to protected endpoints must include a valid token in the Authorization header.

Example protected request header:

- Authorization: Bearer <access_token>

5. Sample Code Snippets (Node.js)

Example: User Login Endpoint

```
app.post('/api/auth/login', async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ email });
  if (!user) return res.status(400).json({ message: 'Invalid credentials' });

  const isMatch = await bcrypt.compare(password, user.password);
  if (!isMatch) return res.status(400).json({ message: 'Invalid credentials' });

  const accessToken = jwt.sign({ id: user._id, role: user.role }, process.env.JWT_SECRET, {
expiresIn: '1h' });
  const refreshToken = jwt.sign({ id: user._id }, process.env.JWT_REFRESH_SECRET, {
expiresIn: '7d' });

  res.json({ accessToken, refreshToken });
```

```
});
```

Example: Token Verification Middleware

```
function verifyToken(req, res, next) {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];
  if (!token) return res.sendStatus(401);

  jwt.verify(token, process.env.JWT_SECRET, (err, user) => {
    if (err) return res.sendStatus(403);
    req.user = user;
    next();
  });
}
```

6. Security Best Practices

- Implement CORS policy for allowed origins.
- Apply rate limiting to protect against brute-force attacks.
- Use Content Security Policy (CSP) headers.
- Perform regular penetration testing and audits.